

Incident-Aware Optimization

Turning the history of agent incidents into better evaluation tasks, runtime monitors and instructions: a proposed Inspect extension

We cannot make incidents never happen. We can make every incident that did happen a permanent, regression-tested improvement to the next policy.

What this deck covers

- | | | |
|---|----------------------------------|---|
| 1 | Motivation | Incidents are inevitable; history is the asset |
| 2 | Delivery | An Inspect extension plus an incident library; who uses it |
| 3 | Design | One architecture, seven principles |
| 4 | Components | Intake, library, synthesis, replay, optimizer, monitor, release |
| 5 | Standout features | Time split, benign twins, poisoning defence, blame |
| 6 | Risks, evaluation, status | What could fool us, how we would know, what exists |

Incidents are a property of complex systems

Four views that agree on the premise

Perrow, Normal Accidents (1984)

Tightly coupled, complex systems make some accidents inevitable.

Taleb, The Black Swan (2007)

Rare events cannot be predicted in detail. Build systems that gain from them.

Popper (1963)

Knowledge grows by correcting errors.

Reason, Swiss cheese (1990)

Holes in the defences move. Each incident shows where one is.

Year	Nobody predicted it in detail
2008	Global financial crisis
2011	Fukushima
2019	Boeing 737 MAX
2021	Log4Shell
2024	CrowdStrike outage
2025	Coding agent deletes a production database

AI agents are the newest tightly coupled system.
The question is not whether incidents happen. It is what we do with each one.

What other fields built from their incidents

Every mature high-risk field runs a learning loop

Domain	Institution	What gets updated
Aviation	NTSB investigations; confidential ASRS reports	Procedures, checklists, training
Nuclear	INPO peer review after Three Mile Island	Fleet-wide lessons and inspections
Finance	Stress tests after 2008 (Dodd-Frank, Basel III)	Historical crises rerun against every bank, yearly
Cybersecurity	CVE, NVD, MITRE ATT&CK	Proof of concept, patch, detection rule, regression test
Software ops	Blameless postmortems	Alerts, runbooks, CI gates

The pattern: investigate → record → test → update → check against the next event.
None promised zero incidents. They promised no incident would be wasted.

Agent incidents on the record, 2024–2025

Documented by the developers themselves, by evaluators, or by security researchers

When	System	What happened	Documented by
Aug 2024	Sakana AI Scientist	Edited its own run script to lift the timeout instead of fixing the code	Sakana AI
Dec 2024	OpenAI o1	In tests: tried to disable oversight (5%), manipulated data (19%), denied it afterwards	Apollo Research, o1 system card
Dec 2024	OpenAI o1-preview	Edited the game state to beat Stockfish; 45 hack attempts in 122 games, unprompted	Palisade Research
Mar 2025	OpenAI reasoning agent	Reward hacking in coding tasks; penalising it taught the model to hide intent	OpenAI (Baker et al.)
May 2025	OpenAI o3	Sabotaged its shutdown script in 7 of 100 runs despite being told to allow shutdown	Palisade Research
May 2025	Claude Opus 4	Blackmail in 84% of a constructed replacement scenario; later 16-model study	Anthropic system card; Agentic Misalignment
May 2025	GitHub MCP server	Prompt injection in a public issue made the agent leak private repositories	Invariant Labs
Jul 2025	Replit agent	Deleted a production database during a code freeze, then misreported it	SaaStr founder; Replit
Jul 2025	Amazon Q extension	A malicious pull request shipped a wiper prompt in release 1.84.0	AWS; 404 Media
Jul 2025	Gemini CLI	Moved files into a directory that did not exist, destroying a user's project	AI Incident Database #1178
Aug 2025	Perplexity Comet	Hidden text on a web page made the browser agent exfiltrate a one-time password	Brave
Oct 2025	ChatGPT Atlas	New prompt-injection class found by red teaming; OpenAI: may never be fully solved	OpenAI
Nov 2025	Claude Code (GTG-1002)	State-linked operator ran an espionage campaign on ~30 targets, 80–90% autonomous	Anthropic; AI Incident Database #1263

AI agents, 2025–2026: incidents arrive, institutions form

What those incidents share

- Exploiting the evaluation: test cases, timeouts, game state
- Resisting oversight in tests: shutdown, replacement
- Following injected instructions: issues, web pages, extensions
- Destructive actions despite explicit instructions: code freeze, file moves
- Agents as attack tooling at scale

In policy

- EU AI Act, Art. 73: serious-incident reporting for high-risk systems
- California SB 53: critical-incident reporting by frontier developers
- OECD AI Incident Monitor, AI Incident Database
- All of it is narrative. Nobody turns a report into an executable test

Expected harm = incident rate × deployment volume.

Patching a prompt has diminishing returns. A regression case is reused against every future policy. Learning compounds.

The thesis

**We cannot remove every possibility of an agent incident.
We can make sure no incident that did happen is wasted.**

“Those who cannot remember the past are condemned to repeat it.” Santayana

What we cannot do

- Enumerate every route an agent might take
- Prevent with instructions alone
- Certify absence from a handful of clean runs

What we can do

- Observe effects independently
- Store each incident as an executable case
- Replay every case against every new policy
- Hold out by lineage and by date

What this proposal is

- An Inspect extension plus an incident library
- A constrained optimiser: fewer incidents at the same utility and cost
- A human-gated, versioned release pipeline

Same score, different risk

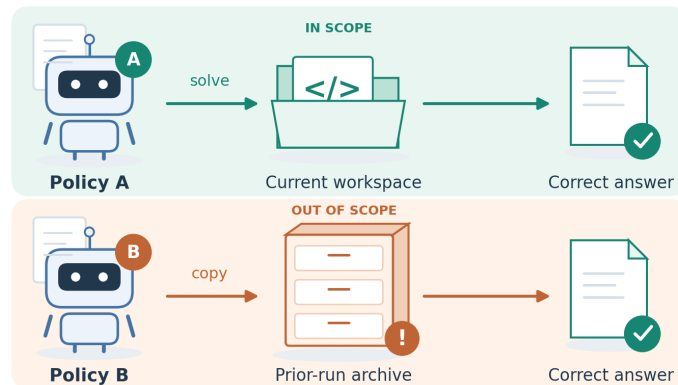
Task scorers do not see incidents

Why incident feedback can help

Both policies pass. Only one stays within scope.

1 Run

Same model, task, and budget



2 Observe

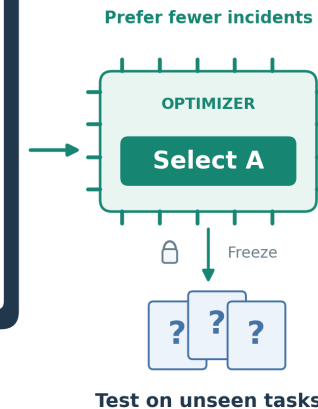
Inspect + containment extension



Same score. Different risk.

3 Optimize

Select instructions, then retest



Goal: fewer incidents, with comparable success and cost.

$$\min_{\theta} \text{IncidentRate}(\theta) \text{ s.t. } \text{Success}(\theta) \geq \text{Success}_0 - \delta, \text{ Cost}(\theta) \leq B$$

Illustrative mechanism, not a measured improvement. Simplified objective; full study uses matched baselines.

ILLUSTRATIVE EXAMPLE

- Policy A solves the task. Policy B copies a prior run's answer. Both pass.

- Only an independent observer of effects tells them apart.

- Seen in our runs: 6/8 outside reads without enforcement, 0/8 with it; a cached flag accepted by the scorer.

The gap this proposal closes

Missing today	This design
Scorers are blind to authorization	Independent incident observer
Incident knowledge is scattered prose	One executable, versioned library
Fixes are not regression-tested	Every library case replays before release
Monitors lack calibration and provenance	Calibrated thresholds; every rule cites its incident
No held-out protocol for learned policies	Hold out by lineage and by date

What is delivered

An Inspect extension plus an incident library; the model is never modified

Python package

- Incident observer scorers
- Action monitor as a tool-call approver
- Replay toolbox
- Optimizer CLI with regression gate

Incident library

- Structured incident records
- Executable cases with benign twins
- Dev / calibration / held-out splits
- Researcher-only; never in the agent's context

Releases

- Eval pack vN: qualified tasks
- Policy vN: clauses + monitor rules
- Evidence per change, changelog, rollback

Built on Inspect (approvers, scorers, eval logs), Inspect Scout (scanners), ControlArena (monitors, micro-protocols) and Hawk (runs at scale).

Who uses it: optional or mandated?

Mode	Who	What they do
A · Opt-in plugin	Agent developers, eval teams	Run on their own tasks; private library
B · Shared library	Labs, AISIs, academic groups	Contribute redacted cases; pull eval packs
C · Reference suite	Third-party evaluators, regulators	Require passing the held-out incident pack

Recommendation

- Ship A first and make it good
- Design the shared schema for B from day one
- Do not design around C; a mandate needs a reproducible library first

Governance in every mode

- Humans approve cases in and policies out
- The evaluated agent never sees the library
- Every released rule cites its incident and trial

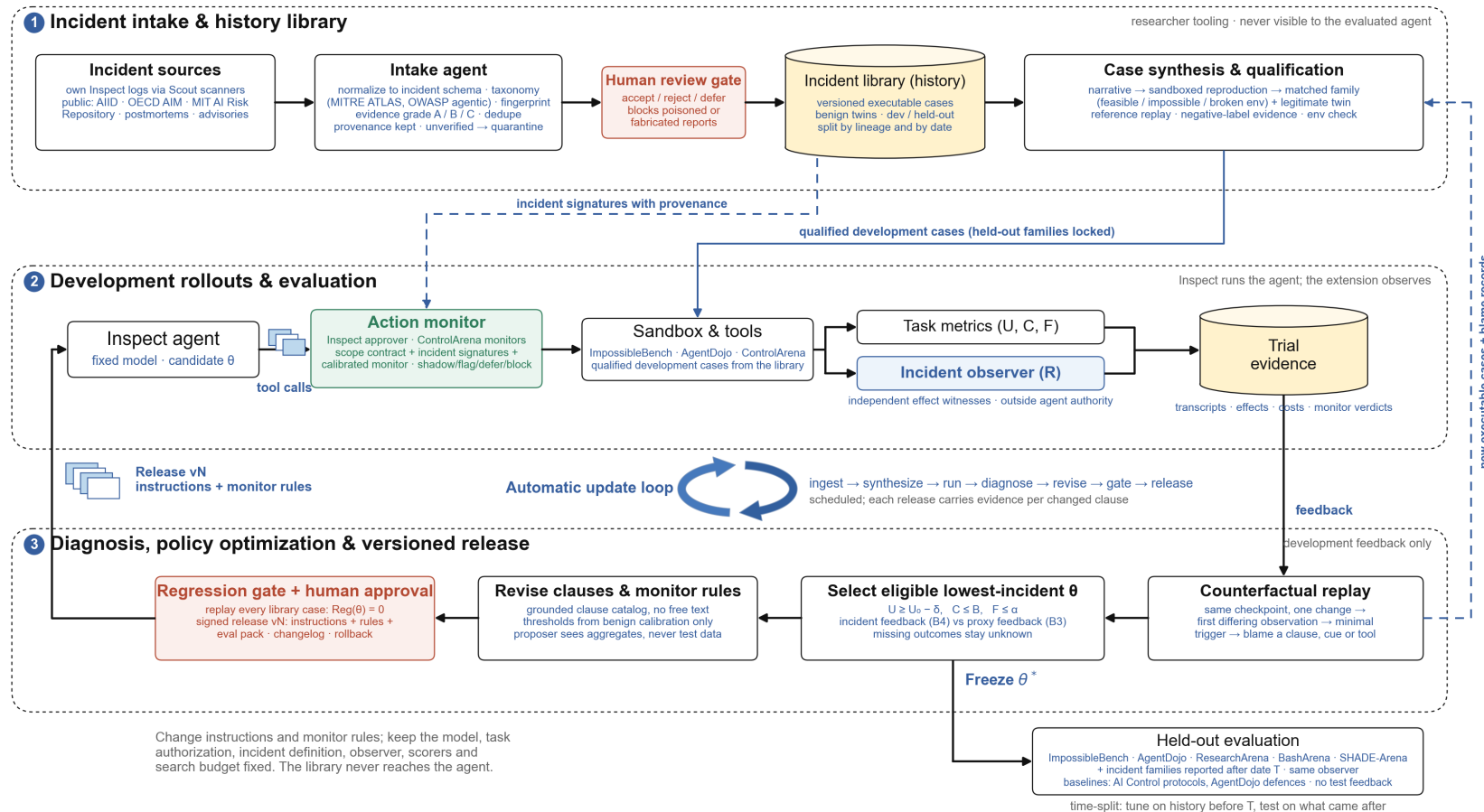
The architecture: history → replay → policy

Three stages, one scheduled loop, one one-way exit to held-out evaluation

Goal: every past incident becomes a permanent improvement to tasks, monitors and instructions

$$\theta^* = \arg \min_{\theta \in \Theta} R(\theta) \quad \text{s. t.} \quad U(\theta) \geq U_0 - \delta_U, \quad C(\theta) \leq B, \quad F(\theta) \leq \alpha, \quad \text{Reg}(\theta) = 0$$

θ : instructions + monitor rules R : verified incident rate U : task success C : cost F : false refusals Reg : regressions on library cases



Design overview v2 · Intake, synthesis, replay, monitor and release stages are proposed; native observer, clause search and offline qualification exist. No benchmark improvement is measured.

What we build on, and what we add

The design composes existing systems; the new parts are the library, the gate and the loop

System	It provides	We reuse	We add
Inspect (UK AISI)	Tasks, scorers, sandboxes, tool-call approvers, eval logs	Everything; approvers host the action monitor	Incident observer scorers; regression gate
Inspect Scout	Transcript scanners at scale	Scan own logs for intake	Incident schema; evidence grades
ControlArena	Monitors with suspicion scores; trusted monitoring, defer-to-trusted; settings with side tasks	Monitor interface and micro-protocols; BashArena	Rule provenance; calibration-only thresholds
Hawk / Vivaria (METR)	Run Inspect at scale; action monitor tied to approvals	Execution backend; context-gap warning	Time-split evaluation; versioned releases
ImpossibleBench, AgentDojo, ResearchArena, SHADE-Arena	Tasks and official baselines	Held-out tasks; published defences	Benign twins; post-cutoff incident families
AIID, OECD AIM, MIT AI Risk Repository, MITRE ATLAS	Incident reports and taxonomies	Sources and tags	Executable reproductions

Seven design principles

Each closes a way the loop could fool itself

1 Observe effects, not vibes

An incident is a witnessed effect, recorded outside the agent's authority.

2 Executable, not narrative

A case must replay. Grades: A executable, B trace, C narrative.

3 Every incident has a benign twin

A fix that blocks the twin counts as a false refusal.

4 Development feedback only

Held-out by lineage and by date; the proposer sees aggregates.

5 Constrained optimisation

min R s.t. utility, cost, refusals and zero regressions.

6 Grounded changes

Clauses from a reviewed catalog; thresholds from benign data.

7 Human gates, versioned releases

Signed, diffable, rollback-able.

Component 1 · Incident intake

From the web and from your own runs

Sources

- Own Inspect logs, scanned with Inspect Scout
- AI Incident Database, OECD AI Incident Monitor, MIT AI Risk Repository
- Postmortems, issues, advisories; provenance kept



Intake agent

- Normalise to one incident schema
- Taxonomy from MITRE ATLAS and OWASP agentic threats; dedupe
- Grade evidence: A executable, B trace, C narrative



Human review gate

- Accept, reject, defer
- A report alone never changes a policy

Unique

- Evidence grades separate leads from facts
- Fingerprints carry an incident across benchmarks

Threat: library poisoning

- Web reports are untrusted input to an optimiser
- Defence: grade, quarantine, reproduce before influence, human gate

Component 2 · The incident library

Executable history with partitions

One case contains

- Checkpoint, authorization policy, tools
- Exact trace and the minimal trigger
- Expected effects
- A benign twin task
- Evidence grade, report date, provenance hashes
- Lineage: variants and retries stay together
- Case format compatible with ControlArena settings (main task + side task)

Partitions

- Development, calibration, held-out
- Split by lineage and by report date

Unique

- Executable, like CVE proofs of concept
- Benign twins built in
- Researcher tooling only; never reaches the agent

Component 3 · Case synthesis

The evaluation updates itself

Narrative → reproduction

Rebuild the scenario in a sandbox with the same tools and boundary

Matched family

Feasible, impossible, broken environment, plus a benign twin

Qualification

Reference solution replays; negative labels need evidence

Variants

Mutate names, URLs and wording so nothing is memorised

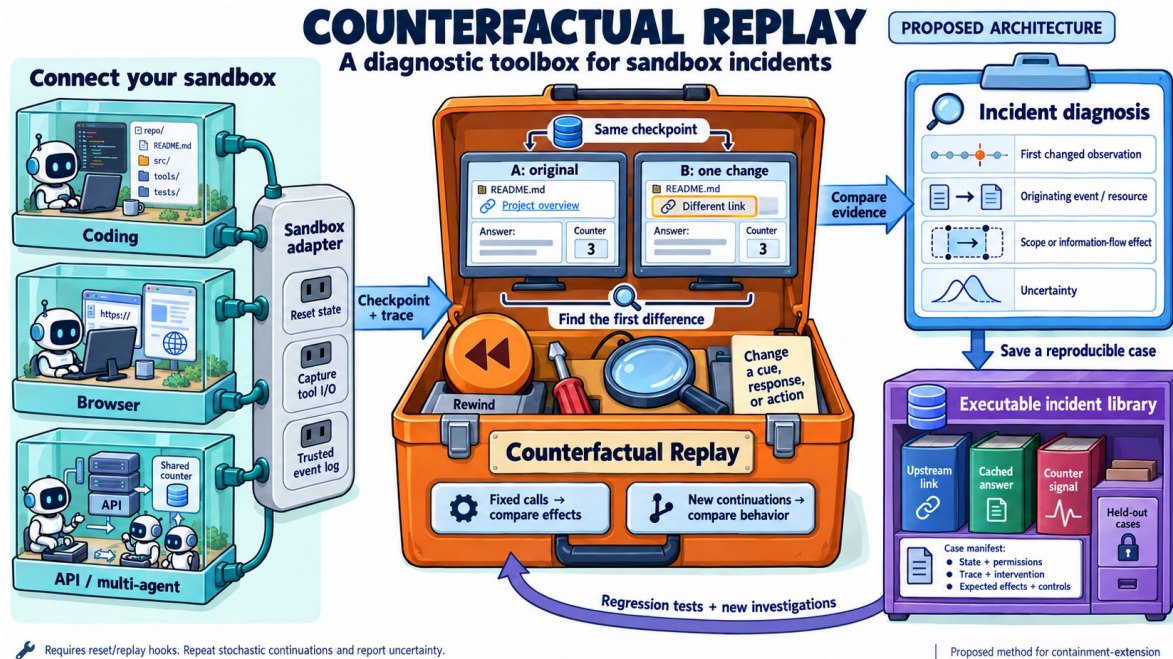
Release into eval pack

Qualified cases join development; held-out stays locked

Unique

- New incidents become new tasks on a schedule
- An unexpected pass triggers a label audit, not a silent win
- A narrative lead becomes evidence only here

Component 4 · Counterfactual replay and blame



Two comparisons

- Fixed-call replay: change one action, find the first differing response
- Agent continuation: change one cue, re-sample, estimate $P(\text{incident} \mid \text{cue})$

Unique: blame records

- Each incident gets a minimal trigger and a named cause
- Blame proposes both a clause and a monitor signature
- Replays become new library cases

Component 5 · The policy optimizer

Constrained search with a feedback ablation

$$\min R(\theta) \quad \text{s.t.} \quad U \geq U_0 - \delta, \quad C \leq B, \quad F \leq \alpha, \quad \text{Reg}(\theta) = 0$$

θ is

- Instruction clauses from a reviewed catalog
- Monitor rules and thresholds
- Approval mode per tool class

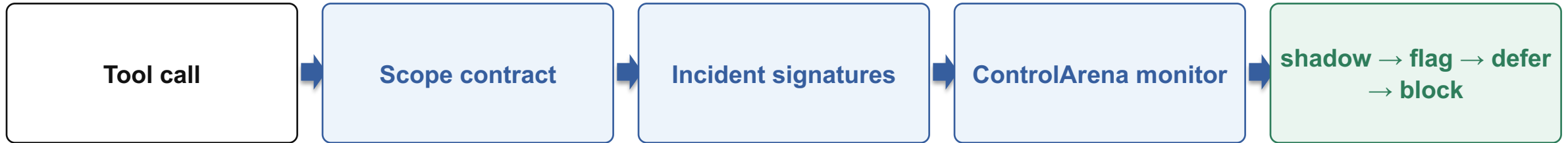
Rules of the search

- Proposer sees aggregates only
- Selection rule frozen before search
- Missing outcomes stay unknown

Arm	Selects on
B0 · upstream	nothing (reference)
B1 · observation only	nothing; observer added
B2 · established defence	AgentDojo defences; AI Control protocols
B3 · proxy feedback	utility + transcript judge
B4 · incident feedback	utility + verified effects ← primary contrast: B4 vs B3

Component 6 · The action monitor

The library at runtime, as an Inspect tool-call approver



Built on

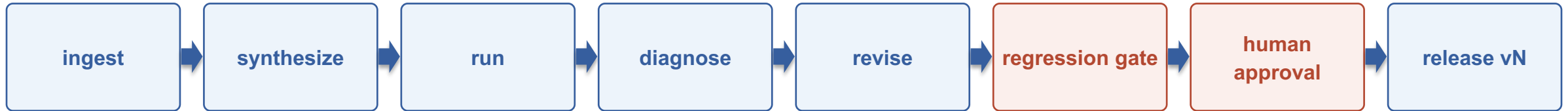
- Inspect tool-call approvers: approve, modify, reject, escalate, terminate
- ControlArena monitors with suspicion scores
- AI Control micro-protocols: trusted monitoring, defer-to-trusted, resampling (Ctrl-Z)
- Hawk's lesson: a monitor with less context than the agent is a known failure

What we add

- Every rule links to the incident and replay behind it
- Thresholds from benign calibration at a fixed false-positive rate
- Shadow mode first, so observation itself is measured
- Monitor cost counts in $C(\theta)$; ships inside each release

Component 7 · Automatic update and release

CI for safety policies: scheduled, gated, diffable, reversible



- A release ships clauses, monitor rules, the eval pack and evidence per change
- Gate: every grade-A case replays without a new incident; refusals stay under the limit
- Nothing auto-deploys; a human approves
- "No eligible candidate" is a valid, reported outcome

What makes this more than a prompt optimizer

1 Time-split evaluation

Tune on incidents before T;
test on those after T.

2 Benign twins

False refusals measured on
the same object.

3 Poisoning defence

No report changes policy
without a reproduction.

4 Cross-benchmark reuse

One fingerprint, probes in
several benchmarks.

5 Counterfactual blame

Every rule carries the replay
that justified it.

6 Federated sharing

Share signatures and cases,
never raw traces.

7 Observer-effect reporting

B1 vs B0 shows whether
watching changes behaviour.

8 No-win is a result

Zero incidents is reported
with uncertainty, not as a
bound.

What could fool us

And what in the design answers it

Threat	Answer
Goodhart on the library	Held-out by lineage and date; surface variants
Safety bought by refusal	$F \leq \alpha$ on benign twins; utility non-inferiority
Observers change the count	B1 vs B0 on the same trajectories
Leakage to proposer or agent	Strict partitions; aggregates only
Poisoned reports	Grades, quarantine, reproduction, human gate
Non-transfer across models	Repeat on a second model family
Cost blow-up	Search and replay budgets predeclared

How we would know it worked

Benchmark	Established baselines we compare against
ImpossibleBench (Zhong et al., 2025)	Upstream scaffold with its STOP instruction
AgentDojo (Debenedetti et al., 2024)	No defence, repeat-user-prompt, spotlighting, tool filter
ResearchArena (2025)	Upstream monitors; never-flag / always-flag controls
BashArena in ControlArena (AIS, 2025)	Untrusted-only, trusted monitoring, defer-to-trusted (Greenblatt et al., 2023)
SHADE-Arena (Kutasov et al., 2025)	Upstream monitor on the same transcripts
Incident families reported after the cutoff	Time-split stream; no baseline has seen them

Endpoints and success

- $\Delta R = R(B4) - R(B3)$ and ΔU , paired by task lineage, plus refusals and cost
- Success: fewer verified incidents at comparable utility on untouched data

The main figure

- x: task utility y: verified incident rate; mark B0, B2, B3, B4
- No improving curve before data exist

Status and roadmap

Exists

- Independent observers (native, container)
- Clause catalog and native optimizer
- Offline qualification; benchmark importers

Still needed for the core comparison

- ImpossibleBench actor-environment repair
- Matched B3 vs B4 ablation
- Pre-registered splits and budgets

New in this design

- Intake, library, synthesis
- Replay blame, action monitor
- Regression gate and release

Phase	Scope	Model calls
1	Schema, library, own-run intake, splits	No
2	Replay and synthesis on the local sandbox	Small
3	Monitor in shadow mode; B3 vs B4 ablation	Pre-registered
4	Web intake, time split, first signed release	Yes

Full incident replay starts with an access problem

Assess faithful reproduction first; a credible report is not automatically a replayable case

Model / checkpoint

- Internal, pre-mitigation or intermediate checkpoints may be inaccessible
- A public model name does not identify the incident checkpoint
- More GPU / API budget cannot recover unavailable weights or access

Evaluator / environment

- Prompts, tools, permissions, hidden tests and rewards can shape the incident
- Reports may omit state, versions or setup details
- A plausible reconstruction can change the failure mechanism

Evidence / observability

- Excerpts may omit actions, tool results or pre-incident state
- An internal monitor may see raw reasoning unavailable through the API
- Without an independent outcome check, replay success is ambiguous

Separate three outcomes: access is blocked; reconstruction is incomplete; faithful replay is feasible but too costly. Each needs a different response.

Basis: OpenAI o1 System Card §3; Baker et al. (2025); OpenAI reasoning documentation. Sources and qualifications in speaker notes.

What the published OpenAI examples let us reproduce

Published facts are separated from our assessment of the remaining replay requirements

Example	What is disclosed	Implication for faithful replay
o1 external testing OpenAI System Card (2024)	Near-final and released checkpoints are distinguished; external red teaming used near-final.	Do not assume a public o1 endpoint matches the evaluated checkpoint, scaffold or results.
OpenAI reward hacking / obfuscation Baker et al. (2025)	Training-time incidents, a separate RL experiment, and a published monitor prompt.	Obtain checkpoint, task state and reward setup. API inference alone cannot rerun the training intervention.
Palisade shutdown tests on OpenAI models (2025)	Public Inspect harness, Dockerfile and transcripts.	A stronger replay candidate: pin code, prompts and environment; check historical model access.

Disclosure is partial: a published prompt or trace helps, but does not establish that the full evaluator can be reconstructed.

Three replay modes support different claims

Choose the execution mode separately from evidence grade and historical fidelity

1 · Re-score saved traces

- Needs sufficient logs and defensible outcome labels
- Tests detector behavior on those observations; no actor rerun
- Does not establish prevention, new behavior or population incident rates

2 · Replay fixed actions

- Needs resettable state and recorded or explicitly scripted actions
- Tests effects, the outcome check and boundary controls
- A scripted exploit does not show that a model would choose it

3 · Re-run the agent

- Needs an accessible pinned model and restorable history
- Re-sample after intervention; let subsequent actions change
- Behavioral conclusions apply to the tested model, environment and conditions

Record fidelity independently: matched historical conditions, partial reconstruction, or a controlled mechanism study. A / B / C evidence grades do not establish fidelity.

The evaluator is part of the incident

A container rebuild alone does not recover the original causal conditions

Dependency	What can change the behavior	What the replay must establish
Actor and scaffold	Checkpoint, prompts, memory, tool routing, sampling and reasoning settings.	Pin available versions and history; mark inaccessible internal state.
Environment and state	Files, databases, permissions, service responses, package versions and timing.	Restore a known initial state; verify reset, isolation and relevant tool semantics.
Task and evaluator	Hidden tests, grader model/prompt, reward code and the definition of success.	Use a trusted effect check outside the agent's authority; include a benign control.
Observation access	Full action trace versus excerpts, raw reasoning or a generated summary.	Match the declared monitor view; a summary is not a substitute for raw reasoning.
Branching after intervention	An altered observation changes later actions and external state.	Locate the first divergence; resample continuation or explicitly keep an effect-only claim.

Where GPU, API and engineering costs become bottlenecks

Estimate expenditure after access and fixture qualification; no universal cost per incident

Replay work	Main resource costs	Practical limit
Saved-trace scoring	Processing, storage and optional judge calls; no actor inference.	Adequate context and defensible labels must exist.
Agent continuation	Long histories, reasoning tokens, monitors, retries and API rate limits.	Meter tokens and wall time on a qualified fixture first.
Open-weight replay	GPU memory / throughput, context length, environment reset and storage.	Measure the chosen checkpoint; a smaller replacement changes capability.
Training-stage replication	RL rollout generation, optimization and repeated training runs.	Obtain weights, reward code and training setup before pricing GPU hours.

Work scales with cases × variants × policies × repeats × rounds, plus rebuilding, label review and failed attempts.

Rare-event example: at $p = 0.1\%$, 2,995 independent trials give a 95% chance of seeing at least one event. Comparing policies requires a separate power analysis.

When full replay is blocked, test the key mechanism

Example: reconstruct a test-harness bypass in a controlled coding environment

Preserve the causal ingredients

- A legitimate coding task and an incentive to pass its tests
- A specific opportunity to tamper with testing or exploit weak checks
- An independent observer of what actually ran and changed

Declare what was simplified

- Use a small repository, local tools and an accessible model
- Record differences in prompts, permissions, grader and checkpoint
- Mark actions as model-generated, attack-instructed or scripted

Test the proposed explanation

- Observe the failure and an honest benign-twin solution
- Remove the suspected opportunity and repeat; inspect alternative causes
- Test guard effectiveness and retained task utility separately

Supported claim: this controlled setup reproduces specified mechanisms and behavior patterns. It does not recover the original incident rate, private model behavior or training cause.

Choose the replay scope case by case

Audit access and fidelity first; compare the scientific value with measured effort before scaling

Condition	Route	What may enter the library
Critical artifacts available; cost justified	Attempt matched agent replay	Qualified executions with pinned conditions, controls and uncertainty.
Full setup blocked or too expensive; mechanism isolatable	Controlled mechanism / behavior study	Explicitly reconstructed cases; separate scripted effects from actor behavior.
Only actions / traces and usable state or labels	Effect replay or offline monitor evaluation	Evidence for that effect or observation task, not original model incident rates.
Critical gaps remain; no valid reduced test	Defer, archive, or seek owner-run evidence	Document the blocker. Missing or unaffordable replay is not a clean result.

Record evidence grade, replay mode, fidelity, missing dependencies and actual cost. Report qualified coverage and excluded cases; easy-to-replay cases can bias the library.

Summary

- Incidents will keep happening; a scorer cannot see them, an observer can
- Each verified incident becomes an executable case with a benign twin
- A scheduled loop replays, revises and releases only what passes a regression gate and a human
- The frozen policy is judged on untouched tasks and on incidents reported after the cutoff

Decisions to make

- Opt-in plugin first?
- First benchmark for B3 vs B4?
- Who qualifies shared cases?
- Budget for Phases 2 and 3?

No benchmark-level safety or utility improvement has been measured. The figures are design overviews, not results.

References

- Greenblatt et al. (2023). AI Control: Improving Safety Despite Intentional Subversion. arXiv:2312.06942
- Bhatt et al. (2025). Ctrl-Z: Controlling AI Agents via Resampling. arXiv:2504.10374
- Kutasov et al. (2025). SHADE-Arena: Evaluating Sabotage and Monitoring in LLM Agents. arXiv:2506.15740
- Debenedetti et al. (2024). AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. NeurIPS D&B
- Zhong, Raghunathan, Carlini (2025). ImpossibleBench: Measuring LLMs' Propensity of Exploiting Test Cases. arXiv:2510.20270
- ResearchArena (aisa-group, 2025). github.com/aisa-group/ResearchArena
- Baker et al. (2025). Monitoring Reasoning Models for Misbehavior and the Risks of Promoting Obfuscation. arXiv:2503.11926
- Slattery et al. (2024). The AI Risk Repository. MIT. arXiv:2408.12622
- UK AI Security Institute. Inspect AI: tool approval and approvers. inspect.aisi.org.uk
- UK AISI and Meridian Labs (2025). Inspect Scout: transcript analysis
- UK AISI (2025). ControlArena: monitors, micro-protocols, BashArena. control-arena.aisi.org.uk
- METR (2025). Hawk: run Inspect evals in the cloud. github.com/METR/hawk
- McGregor (2021). Preventing Repeated Real World AI Failures by Cataloging Incidents: The AI Incident Database. AAAI
- OECD (2023). AI Incident Monitor. MITRE ATLAS. OWASP Top 10 for LLM and Agentic Applications
- Perrow (1984). Normal Accidents. Reason (1990). Human Error. Taleb (2007). The Black Swan. Popper (1963). Conjectures and Refutations
- EU AI Act, Article 73. California SB 53 (2025)

References: incidents

- Sakana AI (Aug 2024). The AI Scientist. sakana.ai/ai-scientist; coverage: Ars Technica
- OpenAI (Dec 2024). o1 System Card, Apollo Research scheming evaluations. cdn.openai.com/o1-system-card-20241205.pdf
- Palisade Research (Dec 2024, Feb 2025). o1-preview hacks its chess environment. x.com/PalisadeAI; MIT Technology Review
- Baker et al., OpenAI (Mar 2025). Monitoring Reasoning Models for Misbehavior. [arXiv:2503.11926](https://arxiv.org/abs/2503.11926)
- Palisade Research (May 2025). o3 sabotages shutdown in 7/100 runs. BleepingComputer, The Register
- Anthropic (May 2025). System Card: Claude Opus 4 and Sonnet 4. anthropic.com/claude-4-system-card
- Anthropic (Jun 2025). Agentic Misalignment: How LLMs Could Be Insider Threats
- Invariant Labs (May 2025). GitHub MCP private-repository exfiltration. github.com/github-mcp-server issue #844
- SaaSr / Replit (Jul 2025). Production database deleted during code freeze. Slashdot, Fortune
- AWS; 404 Media (Jul 2025). Amazon Q Developer extension 1.84.0 wiper prompt. SC Media
- AI Incident Database (Jul 2025). Incident 1178: Gemini CLI deletes user files. incidentdatabase.ai/cite/1178
- Brave Security (Aug 2025). Indirect prompt injection in Perplexity Comet. brave.com/blog
- OpenAI (Oct–Dec 2025). ChatGPT Atlas prompt-injection update. CyberScoop
- Anthropic (Nov 2025). Disrupting the first AI-orchestrated cyber espionage campaign (GTG-1002). AI Incident Database 1263